

Phase 2: Implementation & Baselines

Baseline

Some research suggests that simple linear models can perform competitively with more complex neural network models in virtual cell gene expression prediction [1, 2]. In this study, I compare three models. As a starting point, I write a **linear baseline model**, which serves as the baseline reference for evaluating more complex approaches. This baseline is used to predict gene expression changes under perturbation and provides a point of comparison against an MLP (neural network model) and the “State” model provided by the Arc Institute, which represents a more advanced method.

The goal of this study is to answer the following question: In virtual cell gene expression prediction on the Jurkat dataset (single-cell RNA sequencing data from Jurkat cells under a Perturb-seq experimental setup), does a simple linear model outperform more complex models?

Although prior literature discusses linear models as baselines, the authors did not perform training on the specific dataset I used (`jurkat.h5`), and their code is not publicly available. Therefore, I implemented the baseline model from scratch.

Results

The model was first trained for an unlimited number of epochs, and I manually stopped training when the training loss continued decreasing while the validation loss started to increase. This process was repeated with different random seeds. After around 30 runs, I observed that in most cases, the validation loss started to increase around epoch 6, while the training loss was still decreasing.

```
Using device: cpu
Epoch 001 | Train Loss: 0.500423 | Validation Loss: 0.442843
Epoch 002 | Train Loss: 0.428233 | Validation Loss: 0.428779
Epoch 003 | Train Loss: 0.416965 | Validation Loss: 0.425699
Epoch 004 | Train Loss: 0.411415 | Validation Loss: 0.421582
Epoch 005 | Train Loss: 0.408355 | Validation Loss: 0.420506
Epoch 006 | Train Loss: 0.407416 | Validation Loss: 0.416251
Epoch 007 | Train Loss: 0.405618 | Validation Loss: 0.420942
```

Based on this observation, I modified the training procedure to:

- Save a checkpoint at epoch 6
- Continue training while printing losses
- Stop training at epoch 10

At seed 74: Training loss decreased from approximately 0.500 to 0.407 and validation loss decreased from approximately 0.442 to 0.416

The final test loss was approximately 0.45.

These results indicate that: The model successfully learns meaningful relationships from the data. Performance stabilizes after several epochs, suggesting convergence.

Overall, the linear model provides a reasonable baseline for this task.

Technical Specification

Environment

- Hardware: CPU
- Programming language: Python
- Software dependencies: PyTorch, NumPy, Matplotlib
- Development tools: VS Code and Jupyter Notebook

Experimental Setup - Dataset Construction and Splitting

- Original data: 21,412 cells * 3051 highly variable genes; 68 perturbations and 1 control (no-perturbation cells)
- Control cells: 12,013 (where `target_gene` = "no-target")
- Perturbated cells: 9,399 (where `target_gene` $\neq$ "no-target")
- Validation perturbation cells: 875, selected from perturbations with more than 60 cells (50 out of 68 perturbations have more than 60 observations), taking 10% from each perturbation group
- Test cells: selected from the remaining data after validation split, taking 10% from each perturbation group (if the group has more than 60 cells)
- Training cells: the remaining cells after removing validation and test sets
- Number of genes used: 3051 (highly variable genes)
- Number of perturbations: 68

Only perturbation groups with sufficient cell counts were included in validation and test sets to ensure stable evaluation.

Experimental Setup - Data Preprocessing

- Gene expression data was log-normalized (provided by the dataset)
- Perturbation labels were converted into integer indices and then to one-hot vectors
- Input features were constructed by concatenating control expression with perturbation vectors

Experimental Setup - Model Architecture

The model is defined as:

$$y = Wx + b$$

Input:

$$x \in \mathbb{R}^{3119} \quad (3051 \text{ gene expression} + 68 \text{ perturbation})$$

Output:

$$y \in \mathbb{R}^{3051}$$

Weight matrix:

$$W \in \mathbb{R}^{3051 \times 3119}$$

Bias:

$$b \in \mathbb{R}^{3051}$$

Hyperparameter & Training Details

- Loss function: Mean Squared Error (MSE)
- Optimizer: Adam
- Learning rate: 1×10^{-3}
- Batch size: 64
- Training samples constructed by pairing a random control cell with each perturbed cell
- Training stopped manually when validation loss increases while training loss decreases
- Checkpoint saved at epoch 6 based on observed validation behavior

Next Steps

The next step of this project is to write an MLP model and run the “State” model for comparison with the linear baseline. Subsequently, hyperparameters will be tuned to assess the sensitivity of each model.

Reference

- [1] C. Ahlmann-Eltze, W. Huber, and S. Anders, “Deep-learning-based gene perturbation effect prediction does not yet outperform simple linear baselines,” *Nature Methods*, vol. 22, pp. 1657–1661, 2025. [Online]. Available: <https://www.nature.com/articles/s41592-025-02772-6>. [Accessed: Mar. 29, 2026].
- [2] Z. Wei et al., “Benchmarking algorithms for generalizable single-cell perturbation response prediction,” *Nature Methods*, vol. 23, pp. 451–464, 2026. [Online]. Available: <https://www.nature.com/articles/s41592-025-02980-0>. [Accessed: Mar. 29, 2026].